

Guide d'utilisation

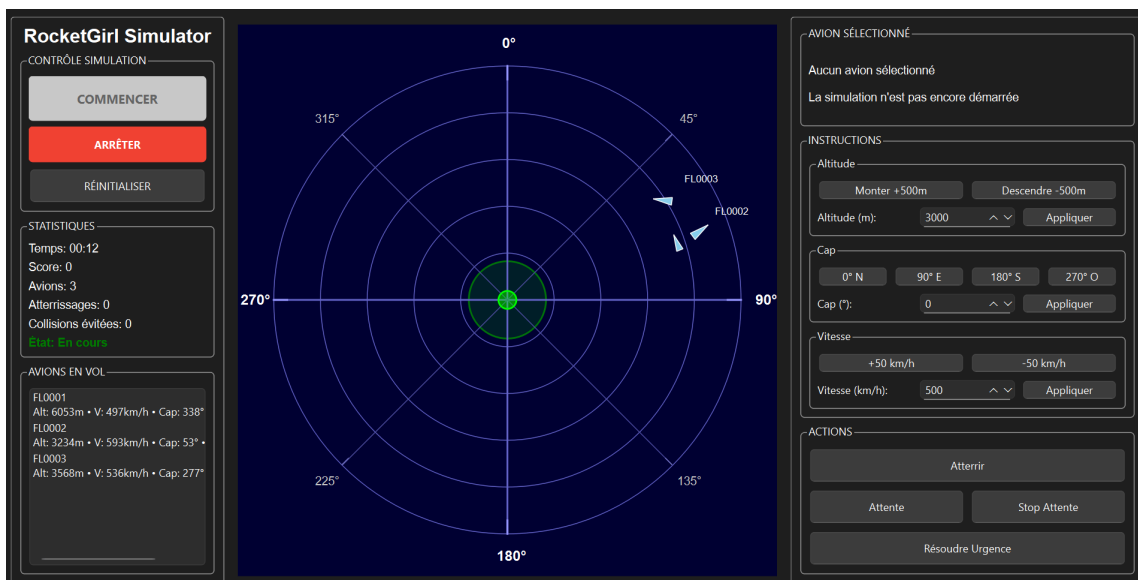
Guide d'utilisation

- Lancer le code via le fichier nommé 'principal.py'.
- La fenêtre d'accueil s'ouvre avec les consignes.
- Appuyez sur "commencer" quand vous êtes prêt.
- Le radar apparaît et les avions apparaissent de façon aléatoire avec des caractéristiques aléatoires.
- Le but est de faire atterrir le plus d'avion possible qui arrive dans votre zone sans accidents ou collisions et en gérant les urgences.
- Pour avoir les informations d'un avion cliquer sur l'avion voulu sur le radar. Les informations apparaissent dans le cadre en haut à droite.
- Pour que l'avion soit en condition d'atterrissage il faut qu'il soit dans la zone verte, que l'altitude ≤ 100 m et la vitesse ≤ 50 km/h.
- Pour avoir ces caractéristiques il faut parfois modifier les caractéristiques de l'avion avec les différents boutons de la colonne de droite.
- Il y a 4 grande actions à accomplir :
 - modifier la vitesse
 - modifier le cap
 - modifier l'altitude
 - gérer des urgences
- Quand toutes les conditions sont remplies, appuyer sur ATERRIR.
- Si trop d'avions sont prêts à atterrir, on peut mettre en attente certains avions il stoppera complètement de bouger. Le joueur perd 5 points toutes les 10 secondes pour chaque avion en attente.
- Il faut aussi régler les urgences (avions qui apparaissent en rouge) le plus vite possible en appuyant sur le bouton. Le joueur perd 1 point par seconde ou l'avion est en situation d'urgence.
- Il faut éviter les collisions entre avions. Voici les conditions de collision à éviter : la distance horizontale (distance sol (x,y)) entre les 2 avions est inférieur à 500 m, et la distance verticale (différence d'altitude) entre les 2 avions est inférieur à 100m. En cas de collision, le joueur perd 500 points.
- Pour que les modifications soient faites jusqu'au bout, il faut garder l'avion sélectionné jusqu'à la fin du changement.

- Il n'y a pas vraiment de condition de victoire ou de fin de jeu. Le but est de faire le plus de points possibles et le plus d'atterrissage possibles en 15min.

Schéma de l'interface graphique

Schéma de l'interface graphique



A gauche : informations sur les avions en vol en bas et juste au-dessus sur le temps de jeu, le nombre de points, le nombre de collision et d'atterrissage, et l'état du jeu. En haut les 3 boutons pour mettre en pause commencer et recommencer le jeu.

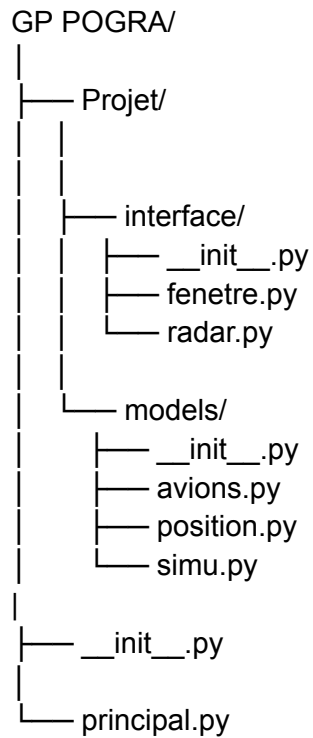
Au milieu : l'écran de la simulation sur lequel les avions sont affichés et depuis lequel l'utilisateur peut choisir l'avion pour modifier ses paramètres.

A droite : le tableau de contrôle. Le joueur peut résoudre un problème, atterrir et mettre en attente un avion après avoir cliqué dessus, via les 3 boutons du bas. Juste au dessus il a la possibilité de modifier l'altitude, le cap et la vitesse des avions soit manuellement en entrant une valeur au clavier soit via des boutons prédéfinis. Et juste au-dessus l'utilisateur peut voir en direct toutes les informations sur l'avion sélectionné.

Explication des modules et bibliothèques utilisés

Explication des modules et bibliothèques utilisés

- **Langage** : Python 3.11+
- **Framework graphique** : PySide6 (Qt for Python)
- **Bibliothèques standard** : random, math, typing, enum, dataclasses



MODULES ET BIBLIOTHÈQUES

Random

- **Import** : `import random`
- **Usage** :
 - Génération aléatoire de positions, caps, vitesses et altitudes des avions.
 - Création d'urgences aléatoires.
- **Fonctions utilisées** :
 - `random.uniform(a, b)` : Valeur flottante aléatoire.
 - `random.randint(a, b)` : Entier aléatoire.
 - `random.choice(seq)` : Élément aléatoire d'une séquence.
 - `random.random()` : Flottant entre 0.0 et 1.0.

Math

- **Import** : `import math`
- **Usage** :
 - Calculs trigonométriques pour les déplacements des avions.
 - Calcul de distances (Pythagore).
 - Conversion angles degrés ↔ radians.
- **Fonctions utilisées** :
 - `math.sin(x)`, `math.cos(x)`
 - `math.sqrt(x)`
 - `math.radians(deg)`

Typing

- **Import** : `from typing import List, Optional`
- **Usage** :
 - Annotations de types pour la clarté du code.
 - Support IDE (autocomplétion, vérification).
- **Types utilisés** :
 - `List[Type]`, `Optional[Type]`

Enum

- **Import** : `from enum import Enum`
- **Usage** :
 - Définition des états des avions et types d'urgence.
- **Enums définies** :
 - **EtatAvion** : EN_VOL, ATTERRI, URGENCE
 - **TypeUrgence** : AUCUNE, CARBURANT, PANNE, METEO

Dataclasses

- **Import** : `from dataclasses import dataclass`
- **Usage** :
 - Définition simple de la classe Position.

PySide6.QtWidgets

- **Import** :
python
Copier

```
from PySide6.QtWidgets import QMainWindow, QWidget,  
QVBoxLayout, QHBoxLayout, QPushButton, QLabel, QListWidget,  
QGroupBox, QSpinBox, QFrame, QMessageBox
```
- **Composants utilisés** :
 - QMainWindow : Fenêtre principale.
 - QWidget : Conteneurs génériques.
 - QVBoxLayout, QHBoxLayout : Disposition des widgets.
 - QPushButton, QLabel, QListWidget, QGroupBox, QSpinBox, QFrame : Éléments d'interface.
 - QMessageBox : Pop-ups (non utilisé dans la version actuelle).

Widget	Rôle	Usage dans le projet
QMainWindow	Fenêtre principale	Classe `FenetrePrincipale
QWidget	Widget générique	Radar personnalisé, conteneurs
QVBoxLayout	Disposition verticale	Organisation des panels
QHBoxLayout	Disposition horizontale	Boutons côte à côte
QPushButton	Bouton cliquable	COMMENCER, Appliquer, actions
QLabel	Affichage texte	Statistiques, infos avion, message accueil
QListWidget	Liste cliquable	Liste des avions en vol
QGroupBox	Regroupement	STATISTIQUES, INSTRUCTIONS
QSpinBox	Saisie numérique	Cap, altitude, vitesse
QFrame	Cadre avec bordure	Panels principaux

PySide6.QtCore

- **Import :**

```
python
Copier
from PySide6.QtCore import Qt, QTimer, QPointF
```
- **Composants utilisés :**
 - Qt : Constantes pour alignement, boutons, focus.
 - QTimer : Rafraîchissement de l'interface (10 FPS).
 - QPointF : Points flottants pour le dessin des avions.

Classe/Constante	Rôle	Usage
Qt	Constantes Qt	Alignement, boutons souris, focus
QTimer	Minuterie	Rafraîchissement à 10 FPS
QPointF	Point flottant	Sommets triangles avions

PySide6.QtGui

- **Import :**
python
Copier

```
from PySide6.QtGui import QPainter, QPen, QBrush, QColor,  
QFont, QPolygonF
```
- **Composants utilisés :**
 - QPainter : Dessin 2D (radar, avions).
 - QPen, QBrush, QColor : Styles de traits et couleurs.
 - QFont : Polices de texte.
 - QPolygonF : Polygones pour les avions.

Classe	Rôle	Usage
QPainter	Dessin 2D	Radar, avions, cercles
QPen	Style de trait	Couleur, épaisseur lignes
QBrush	Remplissage	Couleur intérieure formes
QColor	Couleur RGBA	Toutes les couleurs du jeu
QFont	Police de texte	Labels, angles radar
QPolygonF	Polygone flottant	Triangles des avions

DESCRIPTION DES CLASSES

Classe Position

- **Fichier** : `Projet/models/position.py`
- **Rôle** : Représente une position 3D (x, y, altitude).
- **Attributs** :
 - `x: float, y: float, altitude: float`
- **Système de coordonnées** :
 - Centre (0,0) = Aéroport.
 - x positif = Est, y négatif = Nord.
 - Altitude : 1000 m à 10000 m.

Classe Avion

- **Fichier** : `Projet/models/avions.py`
- **Rôle** : Représente un avion avec sa position, état et comportement.
- **Attributs principaux** :
 - `identifiant: str, position: Position, cap: float, vitesse: float`
 - `etat: EtatAvion, urgence: TypeUrgence, en_attente: bool`
 - `temps_attente: float, temps_urgence: float`
- **Méthodes importantes** :
 - `mettre_a_jour(delta_temps: float)` : Met à jour la position et l'état.
 - `changer_cap()`, `changer_altitude()`, `changer_vitesse()` : Modifie les valeurs cibles.
 - `activer_attente()`, `desactiver_attente()` : Gère l'état d'attente.
 - `atterrir()` : Passe en état atterri.
 - `resoudre_urgence()` : Résout l'urgence.
 - `get_couleur()` : `str` : Retourne la couleur selon l'état.
 - `get_info()` : `str` : Retourne les infos pour l'interface.

Classe Simulation

- **Fichier** : `Projet/models/simu.py`
- **Rôle** : Gère la logique du jeu, les avions, les collisions et le scoring.
- **Attributs principaux** :
 - `avions: List[Avion], score: int, avions_atterris: int`
 - `collisions_evitees: int, simulation_en_cours: bool`
 - `temps_ecoule: float, temps_dernier_spawn: float`

- **Constantes :**
 - SPAWN_BASE_INTERVAL = 60.0 : 1 avion/minute.
 - MAX_AVIONS_SIMULTANES = 6 : Max 6 avions en vol.
- **Méthodes clés :**
 - ajouter_avion_aleatoire() : Génère un avion aléatoire.
 - mettre_a_jour(delta_temps: float) : Cœur de la boucle de jeu.
 - detecter_collisions() : Détecte les collisions (-500 points).
 - demander_atterrissage() : Vérifie les conditions d'atterrissage.
 - mettre_en_attente(), desactiver_attente(), resoudre_urgence() : Gère les actions.

Classe RadarWidget

- **Fichier :** Projet/interface/radar.py
- **Rôle :** Affiche le radar et les avions.
- **Hérite de :** QWidget
- **Méthodes principales :**
 - paintEvent(event) : Dessine le radar, les zones et les avions.
 - dessiner_avion(painter, avion, centre_x, centre_y, rayon_max) : Dessine un avion.
 - mousePressEvent(event) : Gère les clics pour sélectionner un avion.
- **Échelle :** 1 pixel = 30000 m / rayon_radar.

Classe FenetrePrincipale

- **Fichier :** Projet/interface/fenetre.py
- **Rôle :** Interface utilisateur principale.
- **Hérite de :** QMainWindow
- **Composants :**
 - Panel gauche : Contrôles + Statistiques + Liste avions.
 - Panel central : Radar.
 - Panel droit : Infos avion + Instructions + Actions.
- **Méthodes clés :**
 - commencer_simulation() : Démarre la simulation avec 3 avions.
 - mettre_a_jour_simulation() : Met à jour l'interface (10 FPS).
 - activer_controles(), desactiver_controles() : Gère l'état des boutons.

Diagramme UML des classes

Diagramme UML des classes

Position
- x: float - y: float - altitude: float
+ __init__(x, y, altitude)

<<enum>> EtatAvion
+ EN_VOL + ATTERRI + URGENCE

<<enum>> TypeUrgence
+ AUCUNE + CARBURANT + PANNE + METEO

Avion
- identifiant: str - position: Position - cap: float - vitesse: float - altitude_cible: float - vitesse_cible: float - cap_cible: float - carburant: float - etat: EtatAvion - urgence: TypeUrgence - en_attente: bool - temps_attente: float - temps_urgence: float
+ mettre_a_jour(delta_temps: float) + changer_cap(nouveau_cap: float) + changer_altitude(nouvelle_altitude: float) + changer_vitesse(nouvelle_vitesse: float) + activer_attente() + desactiver_attente() + atterrir() + resoudre_urgence() + get_couleur(): str + get_info(): str

Simulation
- avions: List[Avion] - temps_ecoule: float - score: int - avions_atterris: int - collisions_evitees: int - avion_selectionne: Optional[Avion] - aire_atterrissage: Position - prochain_id: int - simulation_en_cours: bool - temps_debut: float - temps_dernier_spawn: float
+ demarrer_simulation() + arreter_simulation() + ajouter_avion_aleatoire() + mettre_a_jour(delta_temps: float) + detecter_collisions() + selectionner_avion(identifiant: str) + donner_instruction_cap(cap: float) + donner_instruction_altitude(delta: float) + donner_instruction_vitesse(delta: float) + demander_atterrissage() + mettre_en_attente() + desactiver_attente() + resoudre_urgence()

RadarWidget (QWidget)
- simulation: Simulation - timer: QTimer
+ paintEvent(event) + dessiner_avion(painter, avion, centre_x, centre_y, rayon_max) + mousePressEvent(event)

FenetrePrincipale (QMainWindow)
- simulation: Simulation - radar: RadarWidget - widgets (boutons, labels, spinboxes)
+ commencer_simulation() + arreter_simulation() + reinitialiser_simulation() + activer_controles() + desactiver_controles() + mettre_a_jour_simulation()

Relations entre Classes

Relation	Type	Description
Position → Avion	Composition	Un avion possède une position
Avion → Simulation	Agrégation	Simulation gère une liste d'avions
Simulation → RadarWidget	Association	Radar affiche la simulation
Simulation → FenetrePrincipale	Association	Fenêtre contrôle la simulation
RadarWidget → FenetrePrincipale	Composition	Fenêtre contient le radar
EtatAvion/TypeUrgence → Avion	Dépendance	Avion utilise les enums